FIAP — Navegando pelo mundo DevOps

Projeto - Cidades ESGInteligentes

TerraMove ESG API

CI/CD com GitHub Actions, Docker e Azure Container Apps

Integrante	RM
Ana Carolina Silva Santana	RM565232
Luiz Gusttavo Teixeira Braga	RM561220
Patrick Roth	RM565596

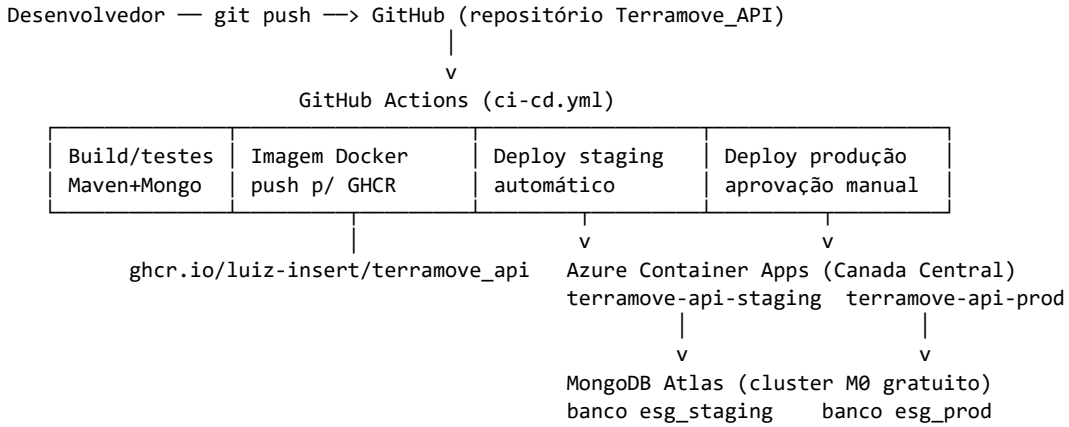
Repositório: https://github.com/Luiz-insert/Terramove_API

Outubro de 2026

1. Visão geral

A TerraMove ESG API é uma API REST em Java 21 com Spring Boot 3.3 e MongoDB. Ela cadastra regiões monitoradas, eventos sísmicos, alertas ambientais e auditorias de compliance (ISO 14001). Nesta fase, a aplicação recebeu um ciclo DevOps completo: containerização, orquestração local com Docker Compose, pipeline de CI/CD e deploy automatizado em dois ambientes na nuvem (staging e produção).

Arquitetura da solução



Ambiente	Onde roda	Banco	Perfil Spring
Local	Docker Compose (PC do desenvolvedor)	Container mongo:7.0 com volume	padrão (seed ligado)
Staging	Azure Container Apps	Atlas — esg_staging	staging (seed ligado)
Produção	Azure Container Apps	Atlas — esg_prod	prod (seed desligado)

2. Pipeline de CI/CD

Ferramenta: GitHub Actions. O pipeline está em `.github/workflows/ci-cd.yml` e é disparado por *push* na branch `main`, por *pull request* e manualmente (*workflow_dispatch*).

Etapas

#	Job	O que faz
1	Build e testes	Sobe um MongoDB 7 como *service container*, configura o Java 21 (Temurin) com cache do Maven e roda `mvn clean verify`: compilação, 6 testes de controller (MockMvc + Mockito), teste de contexto e teste de integração com MongoDB real. Publica o JAR e o relatório de testes como artefatos.
2	Build e push da imagem Docker	Faz o build multi-stage do Dockerfile com Buildx (cache de camadas do Actions) e publica no GitHub Container Registry com duas tags: o SHA do commit e `latest`.
3	Deploy - Staging	Faz login no Azure com um *service principal*, atualiza o Container App de staging para a imagem do commit (`az containerapp update`) e executa um *smoke test* em `/actuator/health`, com novas tentativas por até 5 minutos.
4	Deploy - Produção	Só começa depois que staging passou **e** um revisor aprovou (regra *Required reviewers* do environment `production`). Implanta a **mesma imagem** validada em staging e repete o smoke test.

Lógica do pipeline

- **Falha rápida:** cada job depende do anterior (`needs`). Se um teste falha, nenhuma imagem é publicada e nada é implantado.
- **Pull requests** executam apenas o job 1, para validar o código antes do merge.
- **Imagem imutável:** a tag é o SHA do commit. Staging e produção rodam exatamente o mesmo artefato, o que se confirma pelo campo `version` de `/actuator/info`.
- **Aprovação manual** antes de produção, com registro de quem aprovou.
- **Segredos fora do código:** as credenciais do Azure ficam no secret `AZURE_CREDENTIALS`, e os nomes dos recursos ficam em *variables* do repositório. A URI do banco e a senha do admin são *secrets* do próprio Container App.

Infraestrutura no Azure

O script `infra/azure-setup.sh`, executado uma vez no Azure Cloud Shell, cria o resource group `rg-terramove-esg`, o ambiente do Container Apps, os apps `terramove-api-staging` e `terramove-api-prod` (0,5 vCPU, 1 GiB, escala de 0 a 2 réplicas) e o service principal usado pelo GitHub, com permissão restrita ao resource group.

3. Docker

Arquitetura

Localmente, o `docker-compose.yml` orquestra dois serviços na rede bridge `terramove-network`:

Serviço	Imagem	Configuração
mongodb	mongo:7.0	Usuário e senha via variáveis do `.env`, volume nomeado `mongo_data` para persistência e health check com `mongosh`.
api	terramove-api:local (build do Dockerfile)	Só inicia quando o Mongo está saudável (`depends_on: service_healthy`). Recebe a URI do banco, o perfil e as senhas por variáveis de ambiente.

Dockerfile (multi-stage)

```
# ==================================================================
# Stage 1: Compila o projeto com Maven
# ==================================================================
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app

# Baixa as dependências primeiro (camada fica em cache enquanto o pom.xml não mudar)
COPY pom.xml .
RUN mvn dependency:go-offline -B

COPY src ./src
# Os testes rodam no pipeline de CI; aqui só empacotamos
RUN mvn clean package -DskipTests -B

# ==================================================================
# Stage 2: Imagem final só com o JRE (bem menor que a de build)
# ==================================================================
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Roda como usuário sem privilégios de root
RUN addgroup -S spring && adduser -S spring -G spring

COPY --from=build /app/target/*.jar app.jar
RUN chown spring:spring app.jar
USER spring

ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
  CMD wget -qO- http://localhost:8080/actuator/health || exit 1

ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

Estratégias adotadas

- **Multi-stage:** o Maven e o JDK ficam só no estágio de build. A imagem final contém apenas o JRE 21 Alpine e o JAR.
- **Cache de dependências:** o `pom.xml` é copiado antes do código, então mudanças no código não baixam as dependências de novo.
- **Segurança:** a aplicação roda com o usuário sem privilégios `spring`. O `.dockerignore` exclui `.env`, `.git` e `target/`.

- **Health check** em `/actuator/health` (Spring Boot Actuator), usado pelo Docker, pelo Compose e pelo smoke test do pipeline.
- **12-factor:** toda configuração que muda entre ambientes vem de variáveis de ambiente. A mesma imagem roda em local, staging e produção.

Comandos utilizados

```
cp .env.example .env                # variáveis do ambiente local
docker compose up -d --build        # build da imagem + sobe API e MongoDB
docker compose ps                   # status e health dos containers
docker compose logs -f api          # logs da aplicação
docker compose down -v              # derruba tudo e remove o volume
```

Imagem criada

Registry	Imagem	Tags
GitHub Container Registry (público)	ghcr.io/luiz-insert/terramove_api	<SHA do commit>, latest
Local	terramove-api:local	gerada pelo docker compose

4. Evidências — pipeline

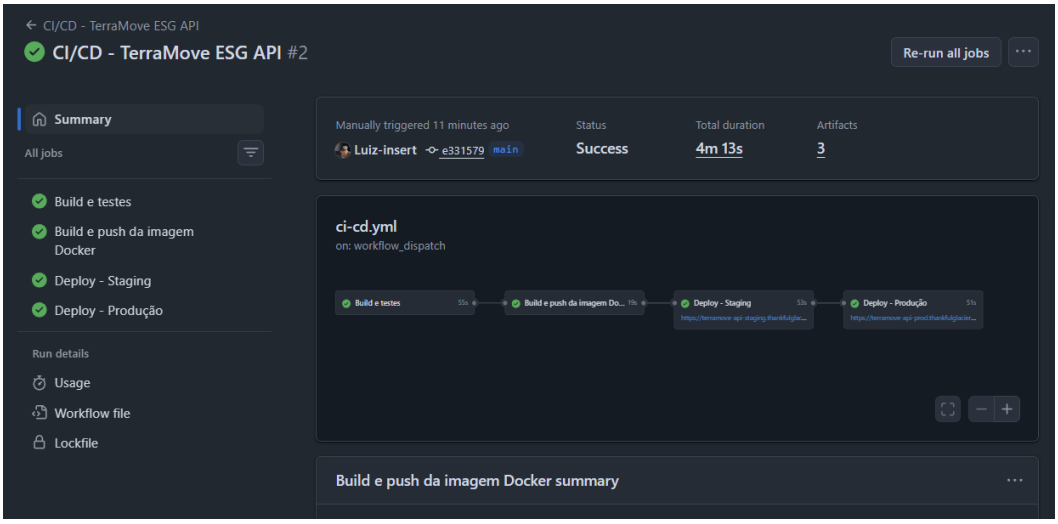


Figura 1 — Execução completa: build e testes, imagem Docker, deploy em staging e em produção, todos com sucesso.

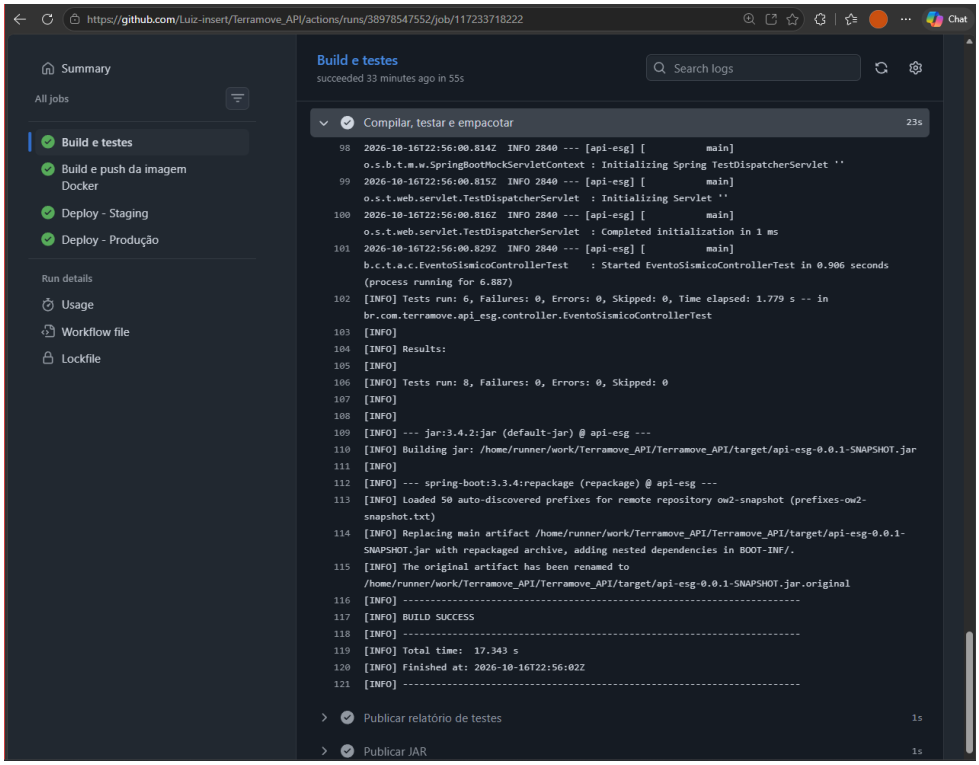


Figura 2 — Job de build: 8 testes executados, 0 falhas, BUILD SUCCESS.

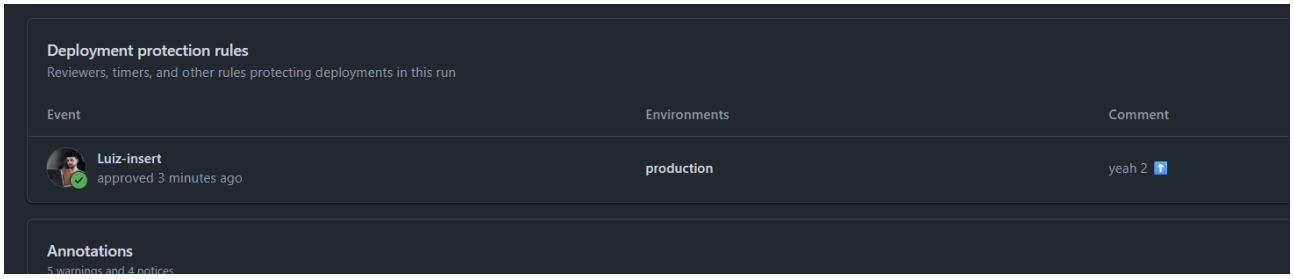


Figura 3 — Aprovação manual exigida antes do deploy em produção.

5. Evidências — ambientes

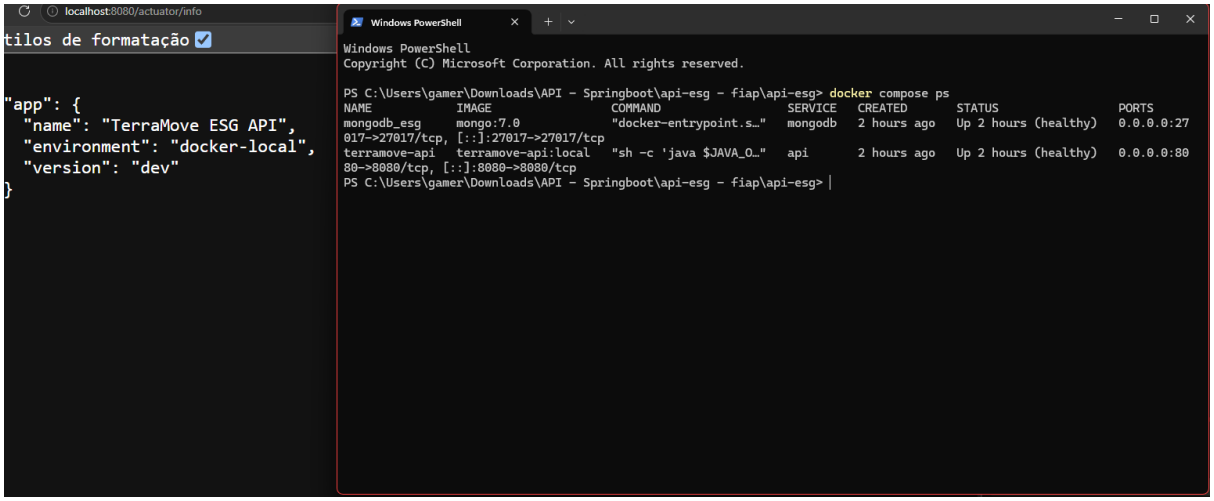


Figura 4 — Ambiente local: containers mongodb_esg e terramove-api saudáveis; /actuator/info respondendo.

Staging

`https://terramove-api-staging.thankfulglacier-03cbae21.canadacentral.azurecontainerapps.io`

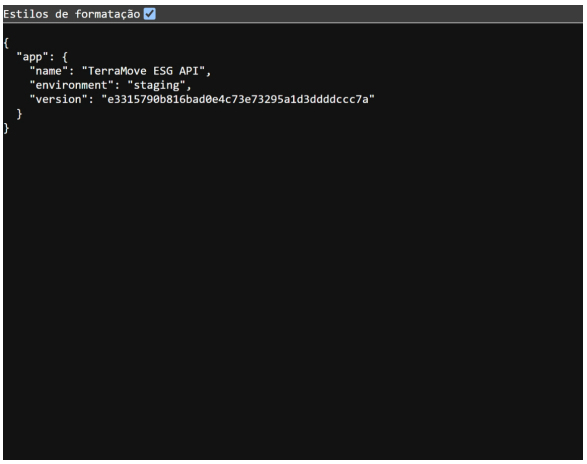


Figura 5 — Staging: /actuator/info indica environment = staging e a versão (SHA) implantada.

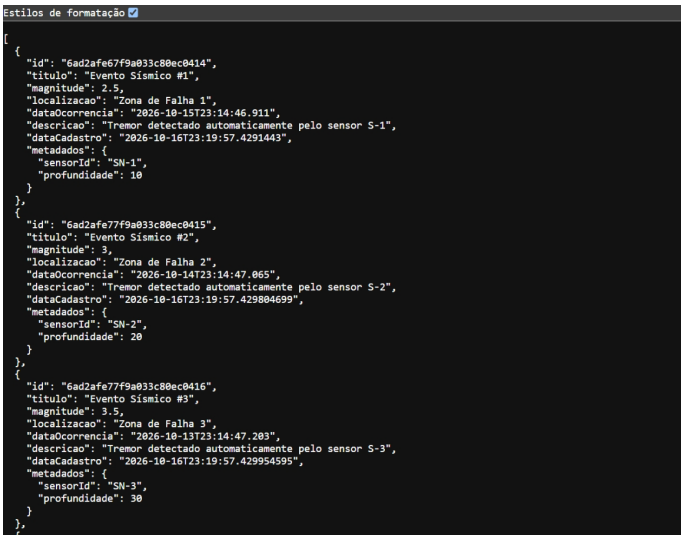


Figura 6 — Staging: GET /api/eventos-sismicos retornando os dados de exemplo.

Produção

`https://terramove-api-prod.thankfulglacier-03cbae21.canadacentral.azurecontainerapps.io`

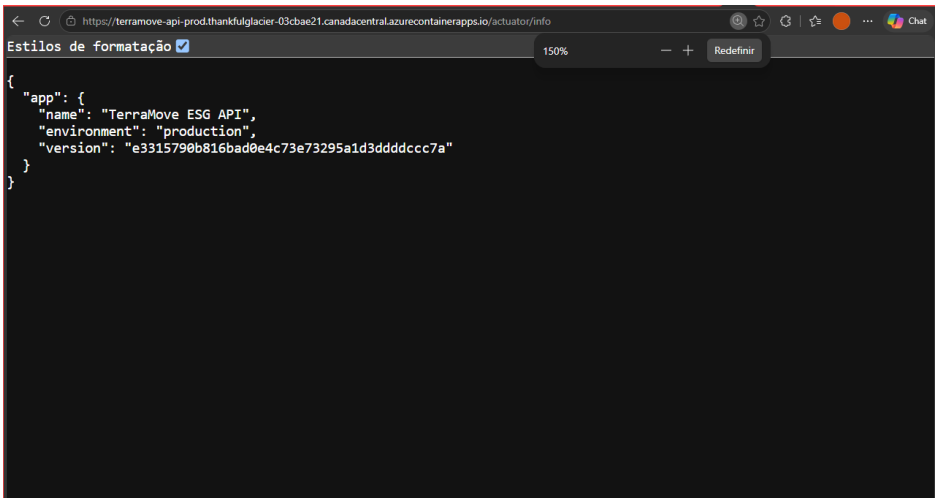


Figura 7 — Produção: environment = production, mesma versão validada em staging.

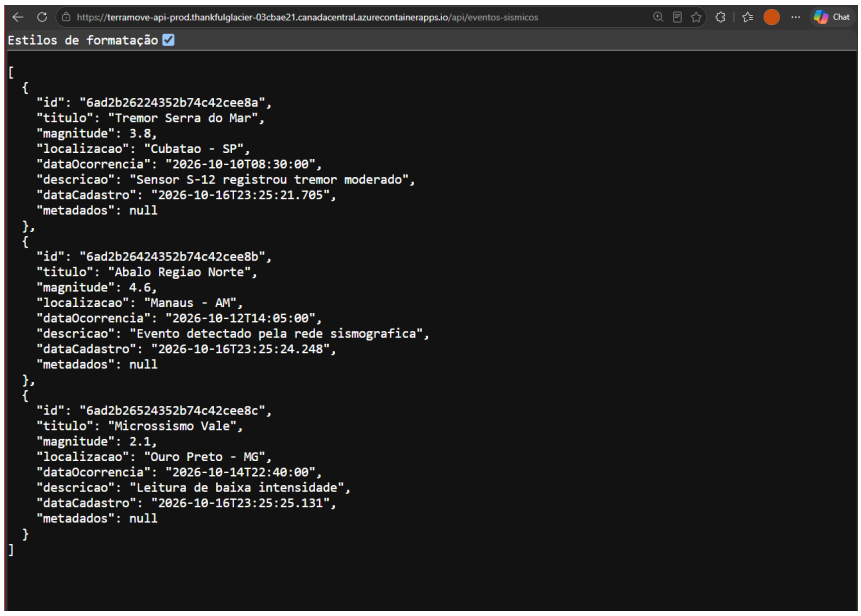


Figura 8 — Produção: eventos cadastrados via POST autenticado (HTTP Basic, usuário admin).

6. Desafios encontrados e soluções

Desafio	Solução
O projeto não compilava com o JDK 23 (Lombok não gerava getters/setters e depois gerava bytecode inválido).	Configuração explícita do Lombok como *annotation processor* no maven-compiler-plugin e atualização para a versão 1.18.36.
O único teste existente dependia de um MongoDB rodando, o que quebraria no CI.	MongoDB como *service container* no GitHub Actions; testes de controller com mocks (rodam sem banco); teste de integração habilitado só quando `MONGODB_URI` existe.
O DataLoader apagava todas as coleções a cada inicialização, o que seria inaceitável em produção.	Perfis `staging` e `prod`; o seed é controlado por `app.seed.enabled` e fica desligado em produção.
Senhas e URI do banco fixas no código.	Externalizadas em variáveis de ambiente, com `.env` local (fora do Git), `.env.example` documentado e *secrets* no Azure e no GitHub.
O Docker Desktop parou de funcionar (I/O error, disco somente leitura) porque o disco virtual estava corrompido.	Diagnóstico pelos logs do Docker Desktop e recriação do disco de dados (docker_data.vhdx).
O MongoDB não subia (exit code 62): o volume antigo tinha sido criado pelo mongo:latest (8.3), incompatível com o 7.0.	Remoção do volume (`docker compose down -v`) e versão do Mongo fixada em 7.0 para builds reproduzíveis.
A assinatura Azure for Students só permite algumas regiões (eastus foi bloqueada por política).	Consulta das regiões permitidas com `az policy assignment list` e uso de canadacentral.
O Azure Cosmos DB estava sem capacidade na região (ServiceUnavailable / high demand).	Troca do banco gerenciado para o MongoDB Atlas (cluster M0 gratuito), com um banco por ambiente. O script de infraestrutura aceita as URIs por variável.
O Container Apps não tem IP de saída fixo, e o Atlas bloqueia IPs desconhecidos.	Liberação de 0.0.0.0/0 no Atlas, mantendo a proteção por usuário e senha do banco.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	Sim
Dockerfile funcional	Sim
docker-compose.yml ou arquivos Kubernetes	Sim
Pipeline com etapas de build, teste e deploy	Sim
README.md com instruções e prints	Sim
Documentação técnica com evidências (PDF ou PPT)	Sim
Deploy realizado nos ambientes staging e produção	Sim